Jenkins Lab: Your First CI Pipeline with Security Gates

Pipeline as Code, Multibranch, Test Reports, Security Stages and Credentials

Duration: about 2 h 15 **Environment:** Linux, macOS or Windows with Docker ≥ 24, a browser, Git, and the `devsecops-lab` project from Labs 1 and 2 on GitLab.

1 Your mission

In Labs 1 and 2 the pipeline ran on GitLab CI. Many companies run Jenkins instead, the most widely used open source automation server. The ideas are the same, only the tool changes: a Git event triggers a pipeline made of stages, and a failing stage stops the delivery. You work in a pair again (Owner and Partner) and learn five skills:

#	Skill	Part	Why it matters
1	Run Jenkins in a container	Setup	Reproducible and disposable: no manual installation on your machine
2	Pipeline as code (`Jenkinsfile`)	1, 2	The pipeline is versioned and reviewed like any other code
3	Test reports	3	Fast feedback: you see which test failed without reading logs
4	Security stages as gates	4	A finding blocks the build, exactly like in Lab 2
5	Credentials handling	5	Secrets never appear in the repository or in the logs

Keep the commit convention from Lab 1: `type(scope): summary (#issue)`. Create three issues in GitLab: #**8** Add a Jenkins pipeline, #**9** Add security stages, #**10** Use credentials safely.

2 Setup: Jenkins in a container (15 min)

2.1 Build the image

The official Jenkins image has no Python and no scanner. Build your own image from a `Dockerfile`, which is a good use of what you saw in the Docker chapter. Create a folder `jenkins-lab` with this `Dockerfile`:

```
FROM jenkins/jenkins:lts-jdk17
USER root
RUN apt-get update \
 && apt-get install -y --no-install-recommends python3 python3-venv python3-pip curl \
 && rm -rf /var/lib/apt/lists/* \
 && curl -sSL https://github.com/gitleaks/gitleaks/releases/download/v8.18.4/gitleaks_8.18.4
    _linux_x64.tar.gz \
    | tar xz -C /usr/local/bin gitleaks
USER jenkins
```

The last line switches back to the unprivileged user `jenkins`: the server never runs as root. On a Mac with an Apple chip, replace `linux_x64` with `linux_arm64` in the URL.

2.2 Run it

```
docker build -t jenkins-lab .
docker run -d --name jenkins -p 8080:8080 -v jenkins_home:/var/jenkins_home jenkins-lab
docker exec jenkins cat /var/jenkins_home/secrets/initialAdminPassword
```

Open `http://localhost:8080`, paste the password, choose *Install suggested plugins*, and create your

own admin user. Only port 8080 is published, and nobody outside your machine should reach it: a Jenkins server can run commands, so exposing it is dangerous.

2.3 Check that your data survives

The volume `jenkins_home` holds everything Jenkins stores: jobs, users, credentials. Create any dummy job, then run:

```
docker rm -f jenkins
docker run -d --name jenkins -p 8080:8080 -v jenkins_home:/var/jenkins_home jenkins-lab
```

Your job is still there.

Q1. Why do we mount a volume on `/var/jenkins_home`? What would you lose without it?

3 Part 1: Your first pipeline (20 min)

3.1 Inline pipeline

Choose *New Item*, name it `hello-pipeline`, type *Pipeline*. In the *Pipeline* section paste:

```
pipeline {
  agent any
  stages {
    stage('Hello') {
      steps { echo 'Hello from Jenkins' }
    }
    stage('Tools') {
      steps { sh 'python3 --version && git --version && gitleaks version' }
    }
  }
}
```

Save, click *Build Now*, and open the build: the *Stage View* shows one box per stage, and *Console Output* shows what each command printed. `agent any` means "run on any available machine", here Jenkins itself.

3.2 Make it fail

Add a stage between the two that runs `sh 'exit 1'`. Build again.

Q2. What happens to the stages after the failing one, and why is this useful in a delivery pipeline?

4 Part 2: Pipeline as code (30 min)

Clicking in a web page is fine to learn, but a pipeline defined in the UI cannot be reviewed, versioned or reproduced. The industry practice is a file named `Jenkinsfile` at the root of the repository.

4.1 Give Jenkins access to GitLab (Owner)

In GitLab, create a personal access token (User settings, Access tokens) with only the `read_repository` scope. In Jenkins, go to *Manage Jenkins*, *Credentials*, add a credential of

kind *Username with password*: your GitLab username, the token as password, ID `gitlab-token`. Never paste the token into a repository URL or a file.

4.2 Write the Jenkinsfile (Owner)

Create the branch `feature/8-jenkinsfile`. Add `.venv/` and `report.xml` to `.gitignore`, and create `Jenkinsfile`:

```
pipeline {
  agent any
  options { disableConcurrentBuilds(); timeout(time: 15, unit: 'MINUTES') }
  stages {
    stage('Setup') {
      steps { sh 'python3 -m venv .venv && .venv/bin/pip install --quiet pytest' }
    }
    stage('Test') {
      steps { sh '.venv/bin/pytest -v' }
    }
  }
}
```

Commit with `ci(jenkins): add Jenkinsfile (#8)`, push, and open an MR with `Closes #8`. GitLab CI and Jenkins now both run on the same project, which is fine for this lab.

4.3 Create a Multibranch Pipeline

In Jenkins, *New Item*, *Multibranch Pipeline*, name `devsecops-lab`. Under *Branch Sources* add a *Git* source: your project URL (`https://gitlab.com/<owner>/devsecops-lab.git`) and the credential `gitlab-token`. Under *Scan Multibranch Pipeline Triggers* tick *Periodically if not otherwise run* with 1 minute (Jenkins on your laptop cannot receive webhooks from GitLab, so it asks). Save.

Jenkins scans the repository, finds your branch because it contains a `Jenkinsfile`, and builds it. Wait for green, review the MR as in Lab 1 (Partner), merge it, and click *Scan Multibranch Pipeline Now*: `main` appears too. Every new branch will get its own pipeline, which is exactly the one-issue-one-branch rule of Lab 1.

Q3. Give two advantages of keeping the `Jenkinsfile` in the repository instead of in the Jenkins web page.

5 Part 3: Test reports (20 min)

5.1 Publish the results

Create the branch `fix/8-test-report` and change the `Test` stage, then add a `post` section at the end of the pipeline:

```
    stage('Test') {
      steps { sh '.venv/bin/pytest --junitxml=report.xml' }
    }
  }
  post {
    always { junit allowEmptyResults: true, testResults: 'report.xml' }
  }
}
```

`post` runs after the stages, even when one failed. Commit with `ci(jenkins): publish test report (#8)`, push, and open the build: a *Test Result* page lists every test.

5.2 Break a test on purpose (Partner)

On a branch `fix/8-failing-test`, change the expected value in `test_app.py` to `"Hello, Anna!"` and push. The build turns red and *Test Result* names the failing test and the two values compared. Fix the test, push again, and watch the build turn green. After a few builds, the page shows a trend graph.

6 Part 4: Security stages as gates (25 min)

You now add the same controls as in Lab 2, in Jenkins syntax. The `gitleaks` binary is already in your image.

6.1 Add the stages (Owner)

Create `feature/9-security-stages`. Add `requirements.txt` with an old dependency, and the Semgrep rules file from Lab 2:

```
requests==2.19.0
```

```
rules:
  - id: no-shell-true
    languages: [python]
    severity: ERROR
    message: "subprocess with shell=True allows command injection (CWE-78)"
    pattern: subprocess.$FUNC(..., shell=True, ...)
```

Update `Setup` to install the scanners, and add a `Security` stage whose three scans run **in parallel**:

```
    stage('Setup') {
      steps { sh 'python3 -m venv .venv && .venv/bin/pip install --quiet pytest pip-audit==2.10.1
    semgrep==1.180.0' }
    }
    stage('Security') {
      parallel {
        stage('Secrets') {
          steps { sh 'gitleaks detect --source . --redact --no-banner' }
        }
        stage('SAST') {
          steps { sh '.venv/bin/semgrep scan --config .semgrep/rules.yml --error --metrics=off --
    exclude=.venv .' }
        }
        stage('Dependencies') {
          steps { sh '.venv/bin/pip-audit -r requirements.txt' }
        }
      }
    }
```

Commit with `ci(jenkins): add security stages (#9)` and push. The `Dependencies` stage turns red because of `requests==2.19.0`, while the other two stay green. Read the table in the console, choose a version at or above the highest fix version listed (at the time of writing, 2.33.0), and commit `security(deps): bump requests to 2.33.0 (#9)`. The build turns green.

6.2 Leak a secret (Partner)

Create `test/9-leak-demo` from the updated branch and push a fake token:

```
printf 'GITLAB_TOKEN = "glpat-abcdEFGH1234ijklMNOP"\n' > config.py
git add . && git commit -m "test(secrets): simulate a leaked token (#9)"
git push -u origin test/9-leak-demo
```

The `Secrets` stage turns red. As in Lab 2, deleting the file is not enough, because the token stays in the history: go back with `git reset -hard HEAD~1`, then `git push -force-with-lease`, and in a real incident revoke the token first. Close the branch without merging.

6.3 Bonus: SAST

Add a function that calls `subprocess.run("echo " + name, shell=True)` on a test branch and watch the `SAST` stage fail. Fix it by passing a list of arguments, as in Lab 2.

7 Part 5: Credentials without leaks (10 min)

Create the branch `feature/10-credentials`. In Jenkins, add a credential of kind *Secret text*, ID `demo-token`, with any dummy value. Add this stage to the pipeline and push:

```
stage('Secret demo') {
  steps {
    withCredentials([string(credentialsId: 'demo-token', variable: 'TOKEN')]) {
      sh 'echo "Token is set, length: ${#TOKEN}"'
      sh 'echo "$TOKEN"'
      sh 'echo "$TOKEN" | rev'
    }
  }
}
```

Note the single quotes: the shell, not Groovy, reads the variable, so the secret is never copied into the pipeline text. In the console, Jenkins replaces the second line with `****`. Look at the third line.

Q4. Jenkins hides the secret in the log, yet the reversed text appears in clear. What does this tell you about masking, and what is the real protection?

8 Wrap-up: Jenkins and GitLab CI (10 min)

You built the same pipeline twice. Complete this table in your own words:

Idea	GitLab CI (Lab 2)	Jenkins (this lab)
Pipeline file	`.gitlab-ci.yml`	`Jenkinsfile`
A unit of work	job	stage
Where it runs	runner and `image`	`agent`
Secrets	CI/CD variables	Credentials store
Parallel work	jobs in the same stage	`parallel` block
Blocking a merge	“Pipelines must succeed”	to be completed

Q5. Name one advantage of GitLab CI and one of Jenkins. Which would you choose for a small team, and why?

9 Deliverables and grading (20 pts)

Submit the project link and screenshots of: the Jenkins login page running from your own image; the *Stage View* of `hello-pipeline` green and red; the Multibranch job listing your branches; the *Test Result* page with a failure and after the fix; the `Security` stage with the three parallel scans, red then green; the masked token in the console; plus your answers to Q1 to Q5.

Jenkins running from a custom image, data kept in a volume	3
First pipeline, success and failure understood	3
`Jenkinsfile` in the repository, Multibranch working	4
Test report published, failing test shown and fixed	3
Security stages in parallel, finding shown and fixed	4
Credentials demo and answers Q1 to Q5	3
Total	20

Remember: the pipeline lives in the repository, a failing stage stops the delivery, and a secret belongs in the credentials store, never in a file or a log.